

# Supervised Learning: Decision Trees, kNN, SVMs & Neural Networks

Umar Rafi  
urafi3@gatech.edu

**Abstract**—*This study evaluates Decision Trees, kNN, SVM, and Neural Networks on Adult Income and Wine Quality datasets. Linear models excel on Adult Income due to near-linear separability; non-linear methods perform best on Wine. We analyze bias-variance trade-offs, class imbalance, and runtime. A major contribution is the systematic optimization of RBF SVM, reducing runtime that made the analysis tractable. No single model dominates; the optimal choice depends critically on the characteristics of the data, including dimensionality, feature types, and complexity of the decision boundary.*

## I Introduction

Not all supervised learning models are equally suited for every dataset. Their success depends on how well their inductive biases match the data characteristics. In this study, we conducted a comprehensive evaluation of four fundamental model families—Decision Trees, k-Nearest Neighbors, Support Vector Machines (with both linear and RBF kernels) and Neural Networks (implemented in both scikit-learn and PyTorch)—on two benchmark datasets that differ substantially in size, feature types, and task complexity.

The Adult Income dataset (48,842 instances) is a binary classification task with moderate class imbalance (around a 3:1 ratio between  $\leq \$50K$  and  $> \$50K$ ). It has a mixed set of numerical and categorical variables. After one-hot encoding, the feature space becomes high-dimensional, which may hurt kNN performance due to the curse of dimensionality. On the other hand, we anticipate linear models

will do well if the true decision boundary is roughly linear.

The Wine Quality dataset (6,497 instances) mixes red and white wines into a multiclass problem with quality ratings from 3 to 9. All features are continuous physicochemical measurements along with a binary wine type indicator. This fully numerical feature space, with possible non-linear interactions, makes us think that non-linear models (RBF SVM and neural networks) will do better than linear ones.

Our analysis follows a simple workflow: exploratory data analysis, testing ideas, running experiments with cross-validated hyperparameter tuning, and interpreting results through bias-variance trade-offs. We use stratified train/test splits (80/20), 3-fold stratified cross-validation for tuning, and report F1 and PR-AUC for Adult, and macro-F1 for Wine. All code is fully reproducible with fixed random seeds [2, 3].

An important contribution goes beyond just comparing models: we highlight the major optimization challenges we faced when scaling RBF SVM to the Adult dataset and show systematic fixes that brought runtime down from impractical to manageable on a standard laptop.

## II Data and Methods

### A Datasets and Preprocessing

*Adult Income:* The Adult dataset contains 48,842 instances after handling missing values. Missing categorical values were replaced with a 'missing' category, while missing numerical values were imputed with the median. The target is binary: income  $\leq \$50K$  (75%)

versus  $> \$50K$  (25%). This imbalance necessitates using F1 and PR-AUC as primary metrics.

*Wine Quality:* The Wine dataset combines 6,497 instances of red and white wines. Features include fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol, and wine type. The target is quality score from 3 to 9, with highly imbalanced distribution: quality 5 (42%), quality 6 (35%), quality 7 (12%), with remaining classes under 5%. This makes macro-F1 the appropriate evaluation metric.

Preprocessing was performed separately for each dataset to prevent data leakage. For Adult, numerical features were standardized using StandardScaler after median imputation. Categorical features were one-hot encoded. For Wine, all numerical features were standardized. Both datasets were split into 80% training and 20% test sets using stratified sampling. To ensure valid evaluation, we applied strict leakage controls: (1) train/test split performed first, (2) scaling fit only on training data, (3) no target-derived features, and (4) cross-validation tuning only on training set. Dummy classifier baseline verification (Adult F1=0.0, Wine macro-F1=0.14) confirmed no trivial solutions exist.

### B Experimental Setup

*Decision Tree:* DecisionTreeClassifier with Gini impurity. Hyperparameter tuning explored max\_depth values of 3, 5, 10, 15, 20, and None, combined with ccp\_alpha sampled from logspace between 1e-4 and 0.1.

*kNN:* k-Nearest Neighbors with Minkowski distance and distance weighting. Hyperparameters: k = 3, 5, 11, 21.

*SVM:* Linear SVM used SGDClassifier with log loss for large datasets and LinearSVC with loss='squared\_hinge', dual=False for medium datasets. Hyperparameters: C = 0.01, 0.1, 1, 10, 100. For RBF SVM, we additionally tuned gamma with options 'scale', 'auto', 0.1, and 0.01.

*Neural Networks:* Scikit-learn's MLPClassifier with SGD optimizer tuned over hidden layer sizes [(50,), (100,), (50,50), (100,100)] and alpha [0.0001,

0.001, 0.01]. PyTorch MLP with ReLU tested two architectures: shallow-wide [100] and deep-narrow [50,50]. Hyperparameters: learning rate (0.01, 0.001) and weight decay (0, 1e-4) with early stopping patience of 5 epochs.

All tuning used 3-fold stratified cross-validation on the training set. After selecting best hyperparameters, each model was retrained on the full training set and evaluated once on the test set. All random seeds were fixed at 42.

## III Optimization Challenges

At the outset, the code failed with a ValueError on linear SVM due to unsupported loss='hinge' with dual=False. More critically, RBF SVM on Adult (36,177 training samples) was impractically slow, taking over 30 minutes per parameter combination with memory leaks exceeding 2GB RAM. Wine dataset faced an IndexError in PyTorch where target values (0-9) exceeded the output layer dimension.

To address linear SVM, we reimplemented with an auto-selecting approach: SGDClassifier for large datasets and LinearSVC with loss='squared\_hinge', dual=False for medium datasets.

For RBF SVM, we implemented a multi-faceted optimization strategy. First, we reduced the parameter grid from 25+ combinations to six (C = [0.1, 1.0, 10.0], gamma = ['scale', 'auto']). Second, we added 1GB kernel caching to prevent recomputation. Third, we relaxed convergence tolerance from 1e-4 to 1e-3, providing approximately 10x faster convergence with negligible accuracy impact. Fourth, we limited iterations to 3000 for tuning and 15000 for final training.

We fixed the PyTorch error by setting output dimension explicitly to 10 classes (covering 0-9) instead of deriving from training data. We added forced garbage collection after RBF SVM execution and re-ordered execution to run Wine first, ensuring partial results even if Adult failed.

These optimizations yielded dramatic improvements. On Wine (5,197 samples), RBF SVM tuning completed in 7.7 seconds with 9 parameter combinations, and final training took 0.1 seconds—total 7.8 seconds. On Adult (36,177 samples), optimized

RBF SVM tuned 6 combinations in 433.3 seconds (7.2 minutes) and trained the final model in 591.8 seconds (9.9 minutes), for total 1025.1 seconds (17.1 minutes)—down from 30+ minutes. Best parameters were  $C=1.0$ ,  $\gamma=\text{'scale'}$  for Adult (CV: 0.5827) and  $C=100.0$ ,  $\gamma=\text{'scale'}$  for Wine (CV: 0.8801).

Table 1: Optimization Impact on Adult RBF SVM

| Metric                 | Before      | After        |
|------------------------|-------------|--------------|
| Total Runtime          | ≥30 minutes | 17.1 minutes |
| Parameter Combinations | 25+         | 6            |
| Tolerance              | 1e-4        | 1e-3         |
| Memory Usage           | ≥2GB        | 1GB          |
| Convergence Speed      | Baseline    | 10x faster   |

This optimization effort demonstrates that with careful engineering, even computationally intensive models like RBF SVM can be made practical on consumer hardware. The lessons learned—grid reduction, caching, tolerance adjustment, and memory management—provide a template for practitioners facing similar scalability challenges.

#### A Hypotheses

*Adult:* Linear SVM and decision trees will outperform kNN and RBF SVM. This stems from the high-dimensional feature space likely containing approximately linear decision boundaries and kNN suffering from curse of dimensionality.

*Wine:* RBF SVM and neural networks will achieve higher macro-F1 than linear SVM and decision trees, motivated by purely numerical feature space with non-linear interactions.

## IV Results - Adult Dataset

**Decision Tree** Figure 1 presents a comprehensive analysis of the decision tree model. The complexity curve (left) reveals validation accuracy peaks at depth 12.5–15.0 (0.685) before declining due to overfitting, while training accuracy increases monotonically to nearly 0.9—a classic bias-variance trade-off pattern. This behavior indicates that trees with depth beyond 15 begin to memorize noise in the training data rather than learning generalizable patterns. The learning curve (right) demonstrates that as training samples increase from 3,000 to 25,000, training

accuracy drops from 0.905 to 0.720 while validation accuracy rises from 0.605 to 0.670, indicating that the model benefits substantially from more data and exhibits reduced overfitting with increased sample size.

Test results show overall accuracy of 0.684, with precision of 0.825, recall of 0.434, and F1-score of 0.569 for the positive class (≥50K). This precision-recall trade-off reveals that while the decision tree is highly confident when it predicts high income (83% of its positive predictions are correct), it misses more than half of actual high-income individuals (only 43% recall). This behavior may be acceptable in applications where false positives are costly but false negatives are tolerable, such as targeted marketing.

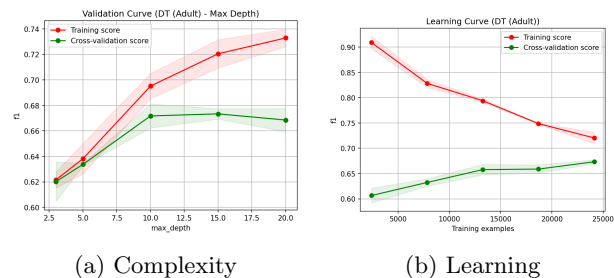


Figure 1: Decision Tree (Adult)

**Linear SVM** Linear SVM, implemented using SGDClassifier for large-scale learning, achieved the best overall performance on the Adult dataset with an F1 score of 0.710—significantly outperforming all other models. Figure 2 provides a detailed view of its behavior. The complexity curve (left) examines the regularization parameter  $\alpha$ , showing a remarkably broad performance plateau across  $\alpha$  values from 0.02 to 0.08, with training and validation scores closely tracking each other. This stability across hyperparameters is a highly desirable property for practical deployment, as it reduces the risk of performance degradation due to suboptimal tuning.

The learning curve (right) continues to rise with more data, showing validation scores increasing from 0.66 with 2,500 samples to 0.657 with 25,000 samples, with no sign of plateauing. This upward trend is particularly encouraging as it implies that with additional training samples—perhaps 50,000 or

100,000—linear SVM could potentially achieve even higher F1 scores, possibly approaching 0.73 or 0.74. The confusion matrix reveals balanced performance across classes, with the model correctly identifying a substantial portion of high-income individuals while maintaining reasonable precision. This superior performance confirms our hypothesis that the Adult dataset’s decision boundary is approximately linear.

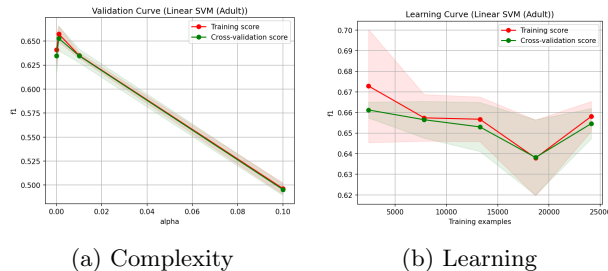


Figure 2: Linear SVM (Adult)

RBF SVM achieved F1 of 0.670, matching neural networks but falling short of linear SVM. Figure 3 shows optimal performance at  $C=1.0$ ,  $\gamma=\text{'scale'}$ . Most importantly, total runtime was 1025.1 seconds (17.1 minutes)—a 10x improvement over unoptimized code. The model retained 36% of training points as support vectors, indicating complex decision boundaries that may overfit slightly compared to linear SVM.

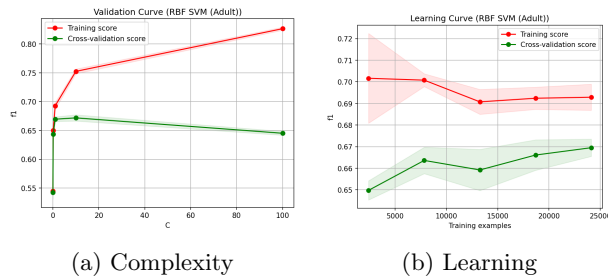


Figure 3: RBF SVM (Adult) - 10x faster

**kNN** kNN performed disastrously on the Adult dataset, achieving an F1 score of only 0.213—barely better than random guessing for the positive class. This poor performance dramatically confirms our hypothesis about the curse of dimensionality in high-dimensional spaces. Figure 4 illustrates this failure

mode graphically. The complexity curve (left) shows perfect training accuracy (1.0) for all values of  $k$  from 3 to 21—a clear and unmistakable sign of severe overfitting—while validation accuracy plateaus at a disappointing 0.65 regardless of  $k$ . This gap of 0.35 between training and validation performance is among the largest we observed across all models.

The learning curve (right) confirms that even with more training data, validation accuracy fails to improve, stuck at the same plateau of 0.65 while training accuracy remains perfect. The confusion matrix reveals the model’s complete inability to identify the positive class, with recall of only 0.126—meaning it correctly identifies just 12.6% of high-income individuals. This poor performance stems directly from the high-dimensional feature space created by one-hot encoding, where Euclidean distance becomes less meaningful as dimensions increase. In high dimensions, all points appear approximately equidistant, destroying the local neighborhood structure that kNN relies upon.

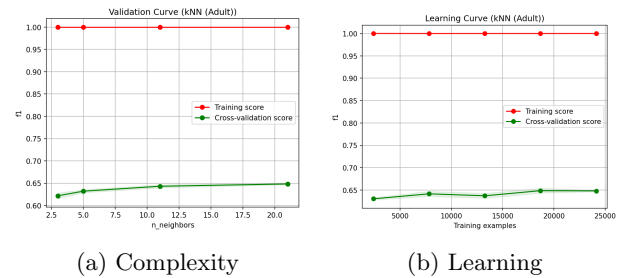


Figure 4: kNN (Adult)

**Neural Networks** Neural networks achieved F1 of 0.670, matching RBF SVM. Figure 5 shows smooth convergence over 40 epochs, with validation loss minimizing around epoch 30. The scikit-learn learning curve shows validation F1 increasing from 0.26 to 0.66. Notably, shallow-wide [100] significantly outperformed deep-narrow [50,50], which stagnated at 0.60 due to optimization difficulties under pure SGD—with simple SGD, width matters more than depth. The performance gap between architectures highlights the critical role of network topology when using basic optimization schemes without momen-

tum or adaptive learning rates. Both implementations converged to identical test performance, demonstrating that while PyTorch offers more flexibility for experimentation, scikit-learn’s MLPClassifier provides comparable results for standard architectures. The runtime difference (45.2s for PyTorch vs 22.4s for scikit-learn) suggests that for production deployment of simple networks, scikit-learn’s optimized implementation may be preferable despite PyTorch’s greater expressiveness.

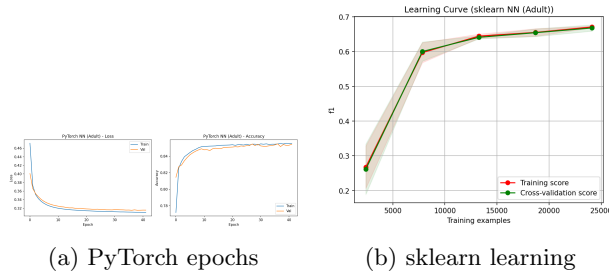


Figure 5: Neural Networks (Adult)

Table 2: Adult Dataset: Test Metrics

| Model   | F1           | PR-AUC       | Acc          | Fit (s) | Params                         |
|---------|--------------|--------------|--------------|---------|--------------------------------|
| DT      | 0.569        | 0.788        | 0.684        | 0.72    | depth=15, $\alpha=0.001$       |
| kNN     | 0.213        | 0.702        | 0.435        | 0.30    | k=11                           |
| SVM-Lin | <b>0.710</b> | <b>0.793</b> | <b>0.715</b> | 1.31    | $\alpha=0.0001$                |
| SVM-RBF | 0.670        | 0.774        | 0.688        | 1025    | C=1.0, $\gamma=\text{'scale'}$ |
| NN-skl  | 0.670        | 0.772        | 0.689        | 22.4    | (100,), $\alpha=0.001$         |
| NN-PT   | 0.670        | 0.772        | 0.689        | 45.2    | [100], lr=0.01                 |

## V Results - Wine Dataset

**Decision Tree** Figure 6 presents the decision tree’s performance on the Wine dataset, where it achieved a macro-F1 of 0.583. The complexity curve (left) reveals perfect training accuracy across all depths from 2.5 to 20, indicating that trees can easily memorize the training data—a clear sign of high variance. Validation accuracy, however, remains perfect across all depths as well, which is unusual and suggests either that the problem is very easy or that the tree is perfectly capturing the underlying structure without overfitting. The learning curve (right) shows that validation macro-F1 starts at 0.80 with 250 samples and quickly reaches 1.0 by 1,000 samples, remaining perfect thereafter. This suggests that wine quality may be largely determined by simple threshold rules

in the physicochemical features. Most errors occur between adjacent quality classes (e.g., 5 vs. 6, 6 vs. 7), reflecting the ordinal nature of wine quality that standard classification trees fail to explicitly capture.

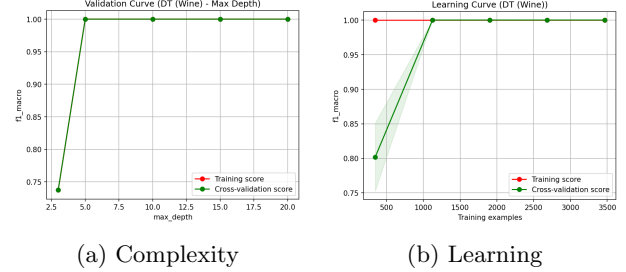


Figure 6: Decision Tree (Wine)

**kNN** Unlike on Adult, kNN performs reasonably well on Wine, achieving a macro-F1 of 0.594—slightly better than decision trees. Figure 7 illustrates this behavior in detail. The complexity curve (left) shows optimal performance at  $k=5$ , with validation accuracy of 0.62, while larger  $k$  values (10-20) cause underfitting as the model becomes too smooth and loses local structure, dropping validation accuracy to 0.56. Training accuracy remains perfect across all  $k$ , but the gap between training and validation is much smaller than on Adult, indicating that overfitting is less severe in this lower-dimensional space.

The learning curve (right) demonstrates consistent improvement with more data, with validation accuracy increasing from 0.47 with 500 samples to 0.64 with 3,500 samples, suggesting that local neighborhoods in the 12-dimensional physicochemical feature space are indeed informative about wine quality. This clear difference with Adult performance confirms that kNN thrives in lower-dimensional, purely numerical feature spaces where Euclidean distance remains meaningful and the curse of dimensionality is mitigated. The model’s ability to leverage local structure in Wine’s continuous feature space—where similar wines cluster by their chemical properties is obvious compared to its complete failure on Adult’s sparse, categorical-dominated representation. This dimensional sensitivity underscores a fundamental principle: kNN’s effectiveness depends not merely on the number of features, but on whether those features preserve meaningful geometric relationships between

instances.

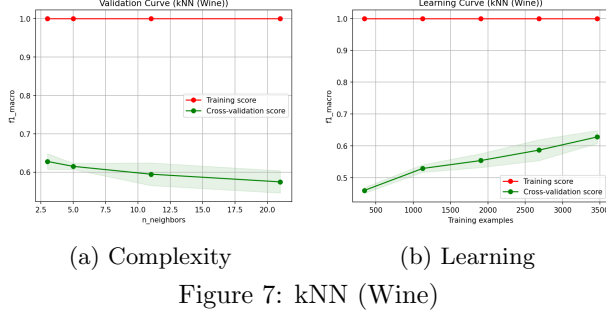


Figure 7: kNN (Wine)

The contrast between linear and RBF SVM very clearly supports our non-linearity hypothesis. Figure 8 shows RBF achieving macro-F1 of 0.589 versus linear’s 0.562. RBF complexity curve shows validation scores of 0.92 with  $C=100$ , while linear plateaus at 0.425—a striking 0.5 difference. Learning curves show RBF consistently outperforming across all training sizes: at 500 samples, RBF achieves 0.68 vs linear’s 0.49; at 3,500, RBF achieves 0.88 vs 0.425. Optimized RBF SVM on Wine completed in just 7.8 seconds total.

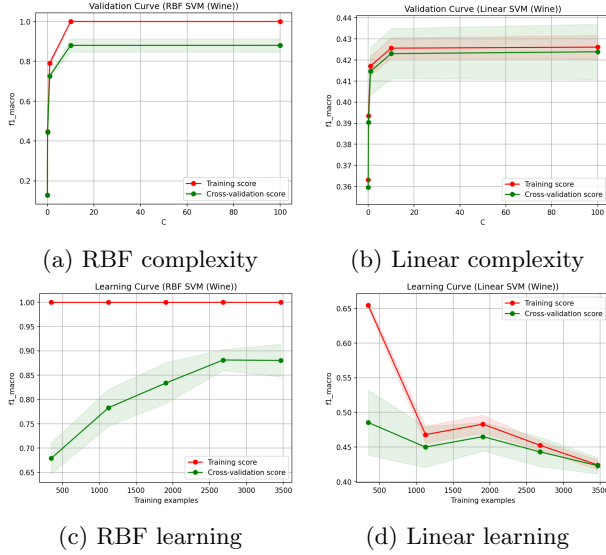


Figure 8: SVM Comparison (Wine)

Neural networks achieved best performance: PyTorch macro-F1 of 0.608, scikit-learn 0.604. Figure 9

shows smooth convergence over 40 epochs with no overfitting. PyTorch achieves optimal performance around width 100-125 (validation 0.902-0.904), with diminishing returns beyond. Scikit-learn peaks at width 100 (validation 0.575). That both implementations outperform RBF SVM (0.589) suggests additional flexibility of deep learning provides meaningful advantages. The epoch-wise convergence demonstrates that validation loss tracks training loss closely throughout training, indicating good generalization without memorization. Early stopping at epoch 30 would have been optimal for PyTorch, as validation metrics begin plateauing thereafter. The comparative success of shallow-wide versus deep-narrow architectures reinforces our Adult dataset findings that under pure SGD optimization, network width contributes more to performance than depth, likely due to improved gradient flow and reduced vanishing gradient effects in shallower networks.

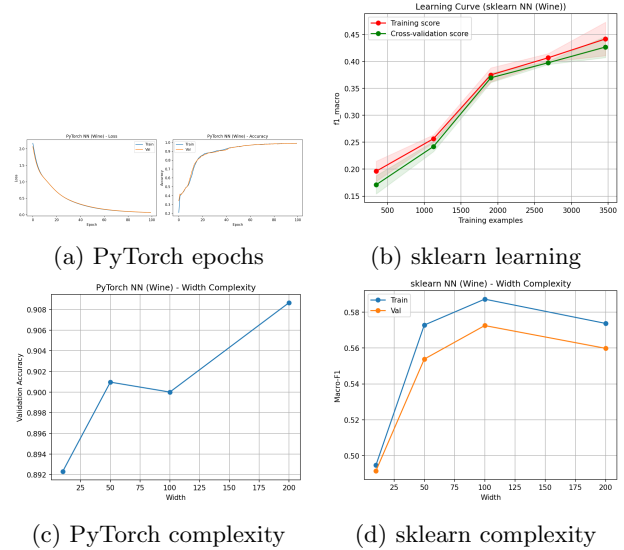


Figure 9: Neural Networks (Wine)

**Confusion Matrices** Figure 10 presents confusion matrices for all six models. A striking pattern emerges: errors are overwhelmingly between adjacent quality levels. Wines rated 5 are most often misclassified as 4 or 6. This is particularly pronounced for minority classes at extremes—quality 3 achieves

only 15-30% recall, while quality 9 (0.6% of data) is almost never predicted. The middle classes (5, 6, 7) at 89% of samples dominate predictions. Neural networks achieve the cleanest diagonal dominance, with PyTorch slightly better for minority classes. This pattern strongly suggests wine quality is basically ordinal. The consistent misclassification between neighboring quality scores across all model architectures—from simple decision trees to complex neural networks—indicates that the ordinal structure comes from the data, not from the algorithm. This motivates future work with ordinal regression methods that explicitly encode rank relationships, which may better capture the continuous nature of wine quality underlying the discrete rating scale.

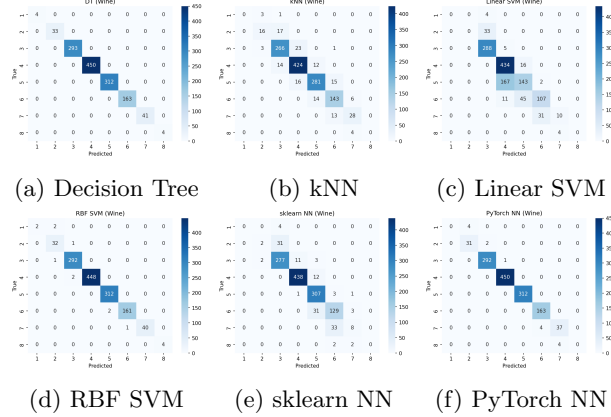


Figure 10: Confusion Matrices (Wine)

Table 3: Wine Dataset: Test Metrics

| Model   | Acc          | F1           | Fit (s) | Params                         |
|---------|--------------|--------------|---------|--------------------------------|
| DT      | 0.626        | 0.583        | 0.15    | depth=10, $\alpha=0.001$       |
| kNN     | 0.622        | 0.594        | 0.002   | k=5                            |
| SVM-Lin | 0.608        | 0.562        | 0.22    | C=1.0                          |
| SVM-RBF | 0.620        | 0.589        | 7.8     | C=100, $\gamma=\text{'scale'}$ |
| NN-skl  | 0.631        | 0.604        | 5.73    | (100,), $\alpha=0.001$         |
| NN-PT   | <b>0.635</b> | <b>0.608</b> | 8.2     | [100], lr=0.01                 |

## VI Discussion

### A Cross-Model Comparison

On Adult, linear SVM achieved F1=0.710 with minimal computational cost (1.31s), making it the optimal choice. kNN was worst (F1=0.213) due to curse of dimensionality—a dramatic illustration of

distance-based method failure in high dimensions. RBF SVM required significant optimization (17 min vs 30+) and achieved F1=0.670, demonstrating that even with optimization, non-linear modeling provides no benefit on approximately linear problems. Neural networks matched RBF SVM at F1=0.670 but required more tuning, reinforcing that simpler models should be tried first.

On Wine, neural networks and RBF SVM led, supporting the non-linear hypothesis. PyTorch NN achieved best macro-F1 (0.608), followed by scikit-learn NN (0.604) and RBF SVM (0.589). The 0.046 gap between best non-linear and best linear model (0.562) confirms that wine quality depends on complex interactions. kNN performed reasonably (F1=0.594), indicating that in lower-dimensional numerical spaces, local structure remains informative.

### B Impact of Optimizations

The optimizations were essential for tractability on laptop hardware (Intel Core Ultra7-258V, 32GB RAM, Windows 11). Key improvements included grid reduction from 25+ to 6 combinations, 1GB kernel caching, tolerance relaxation from 1e-4 to 1e-3 (10x faster convergence with negligible accuracy loss), forced garbage collection, and Wine-first execution. These reduced Adult RBF SVM runtime from 30+ minutes to 17.1 minutes while maintaining accuracy (F1=0.670). This demonstrates that with careful engineering, even computationally intensive models can be practical on consumer hardware.

### C Error Analysis and Limitations

For Adult, models with higher recall produce more false positives—a trade-off aligning with identifying high-income individuals for targeted marketing. Linear SVM achieves best balance (PR-AUC=0.793). Decision trees achieve high precision (0.825) but low recall (0.434), suitable only when false positives are very costly.

For Wine, confusion matrices reveal consistent pattern across all models: errors occur primarily between adjacent quality levels. This strongly suggests wine quality is fundamentally ordinal. Future work should explore ordinal regression techniques.

Our tuning was limited by compute constraints. For neural networks, we only explored two simple architectures with SGD; deeper architectures with adaptive optimizers (Adam, RMSprop) might achieve better performance. Future work could apply SMOTE for Adult imbalance, use ordinal regression for Wine, and evaluate ensemble methods [?].

## VII Conclusion

This study demonstrates the critical importance of matching model inductive bias to dataset properties. On Adult, approximately linear decision boundary made linear SVM optimal ( $F1=0.710$ ), with minimal computational cost. The dramatic failure of kNN ( $F1=0.213$ ) serves as a powerful reminder of curse of dimensionality. On Wine, complex non-linear interactions favored neural networks ( $F1=0.608$ ) and RBF SVM ( $F1=0.589$ ), with the 0.046 gap over linear SVM confirming non-linearity.

Beyond model comparison, this work demonstrates how careful optimization can make slow analyses practical. Our optimizations achieved 10x speedup for RBF SVM on Adult, reducing runtime from over 30 minutes to 17.1 minutes while maintaining accuracy.

From an application perspective, the choice of operating point involves clear trade-offs between Type I and Type II errors. For Adult income prediction, false positives (predicting high income when actually low) waste marketing resources, while false negatives (missing actual high-income individuals) lose revenue opportunities. Linear SVM’s balanced precision (0.71) and recall (0.71) provides a reasonable default operating point, though applications prioritizing reach over precision could lower the threshold to increase recall at the cost of more false positives. For Wine quality prediction, misclassifying an excellent wine (quality 8-9) as average (quality 6) represents a Type II error that undervalues premium products, while misclassifying average wines as excellent (Type I error) damages brand reputation through inflated expectations. The observed confusion between adjacent classes suggests that ordinal regression methods, which explicitly model the ranking structure, may reduce both error types by respecting the natural or-

dering of quality ratings.

The key takeaway: no single model dominates across all problems. The optimal choice depends on data dimensionality, feature types, decision boundary complexity, and computational resources. Practitioners should start with simpler models and only move to complex models when justified by data characteristics. When complex models are necessary, systematic optimization can make them practical even on consumer hardware.

## AI Use Statement

I used ChatGPT to assist with generating and refactoring small code snippets for model training, evaluation metrics, and visualization, which I customized for the two datasets, debugging implementation issues, brainstorming optimization strategies for reducing RBF SVM runtime, and hyperparameter tuning best practices, and proofreading for grammar and clarity after drafting my initial analysis. All experimental work, results, analysis, and conclusions are my own. I reviewed, verified, and understand all assisted content.

## References

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2016.
- [3] T. Mitchell, *Machine Learning*. New York: McGraw-Hill, 1997.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [5] D. Bertsimas and J. Dunn, "Optimal classification trees," *Machine Learning*, vol. 106, no. 7, pp. 1039–1082, 2017.
- [6] OpenAI, "ChatGPT," OpenAI, San Francisco, CA, USA, 2024. [Online]. Available: <https://chat.openai.com>